

Linear Search

Linear search is also called as sequential search algorithm. It is the simplest searching algorithm. In Linear search, we simply traverse the list completely and match each element of the list with the item whose location is to be found. If the match is found, then the location of the item is returned; otherwise, the algorithm returns NULL. It is widely used to search an element from the unordered list, i.e., the list in which items are not sorted. The worst-case time complexity of linear search is $O(n)$.

Algorithm

Linear_Search(a, n, val) // 'a' is the given array, 'n' is the size of given array, 'val' is the value to search

Step 1: set pos = -1

Step 2: set i = 1

Step 3: repeat step 4 while $i \leq n$

Step 4: if $a[i] == val$

set pos = i

print pos

go to step 6

[end of if]

set $i = i + 1$

[end of loop]

Step 5: if pos = -1

print "value is not present in the array "

[end of if]

Step 6: exit

1. Time Complexity

Case	Time Complexity
Best Case	$O(1)$
Average Case	$O(n)$
Worst Case	$O(n)$

Best Case Complexity - In Linear search, best case occurs when the element we are finding is at the first position of the array. The best-case time complexity of linear search is $O(1)$.

Average Case Complexity - The average case time complexity of linear search is $O(n)$.

Worst Case Complexity - In Linear search, the worst case occurs when the element we are looking is present at the end of the array. The worst-case in linear search could be when the target element is not present in the given array, and we have to traverse the entire array. The worst-case time complexity of linear search is $O(n)$.

The time complexity of linear search is $O(n)$ because every element in the array is compared only once.

2. Space Complexity

Space Complexity	$O(1)$
------------------	--------

The space complexity of linear search is $O(1)$.

Binary Search

Binary search is the search technique that works efficiently on sorted lists. Hence, to search an element into some list using the binary search technique, we must ensure that the list is sorted.

Binary search follows the divide and conquer approach in which the list is divided into two halves, and the item is compared with the middle element of the list. If the match is found then, the location of the middle element is returned. Otherwise, we search into either of the halves depending upon the result produced through the match.

```
int binarySearch (int arr[], int l, int r, int x)
```

```
{
    while (l <= r) {
        int m = l + (r - l) / 2;
        // Check if x is present at mid
        if (arr[m] == x)
            return m;

        // If x greater, ignore left half
        if (arr[m] < x)
            l = m + 1;
        // If x is smaller, ignore right half
        else
            r = m - 1;
    }
}
```

```

// If we reach here, then element was not present
return -1;
}

```

Time Complexities

Best case complexity: $O(1)$

Average case complexity: $O(\log n)$

Worst case complexity: $O(\log n)$

Space Complexity

The space complexity of the binary search is $O(1)$.

Bubble Sort

```

1. void bubbleSort(int arr[], int n)
2. {
3.     int i, j;
4.     bool swapped;
5.     for (i = 0; i < n - 1; i++) {
6.         swapped = false;
7.         for (j = 0; j < n - i - 1; j++) {
8.             if (arr[j] > arr[j + 1]) {
9.                 int t=arr[j];
10.                arr[j]=arr[j+1];
11.                arr[j+1]=t;
12.                swapped = true;
13.            }
14.        }
        if (swapped == false)
            break;
    }
}

```

Time Complexities

1. Worst Case Complexity: $O(n^2)$

If we want to sort in ascending order and the array is in descending order then the worst case occurs.

2. Best Case Complexity: $O(n)$

3. Average Case Complexity: $O(n^2)$

Space Complexity

Space complexity is $O(1)$

Optimized bubble sort algorithm is $O(2)$

Bubble Sort is stable and Inplace sorting algorithm.

Selection Sort

Selection sort is a simple and efficient sorting algorithm that works by repeatedly selecting the smallest (or largest) element from the unsorted portion of the list and moving it to the sorted portion of the list.

```
void selectionSort(int arr[], int n)
```

```
{
    int i, j, min_idx;
    for (i = 0; i < n - 1; i++) {
        min_idx = i;
        for (j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx])
                min_idx = j;
        }
        if (min_idx != i)
            swap(arr[min_idx], arr[i]);
    }
}
```

Time Complexities

1. Worst Case Complexity: $O(n^2)$

If we want to sort in ascending order and the array is in descending order then the worst case occurs.

2. Best Case Complexity: $O(n^2)$

3. Average Case Complexity: $O(n^2)$

Space Complexity

Space complexity is $O(1)$

Selection Sort is not stable and Inplace sorting algorithm.

Insertion Sort

```
void insertionSort(int arr[], int n)
{
    int i, key, j;
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}
```

Time Complexities

1. Worst Case Complexity: $O(n^2)$

If we want to sort in ascending order and the array is in descending order then the worst case occurs.

2. Best Case Complexity: $O(n)$

3. Average Case Complexity: $O(n^2)$

Space Complexity

Space complexity is $O(1)$

Insertion Sort is stable and Inplace sorting algorithm.

Divide and Conquer Algorithm

A divide and conquer algorithm is a strategy of solving a large problem by

1. breaking the problem into smaller sub-problems
2. solving the sub-problems, and
3. combining them to get the desired output.

Quick Sort Algorithm

Quicksort is based on the divide-and-conquer strategy. It picks an element as pivot, and then it partitions the given array around the picked pivot element. In quick sort, a large array is divided into two arrays in which one holds values that are smaller than the specified value (Pivot), and another array holds the values that are greater than the pivot.

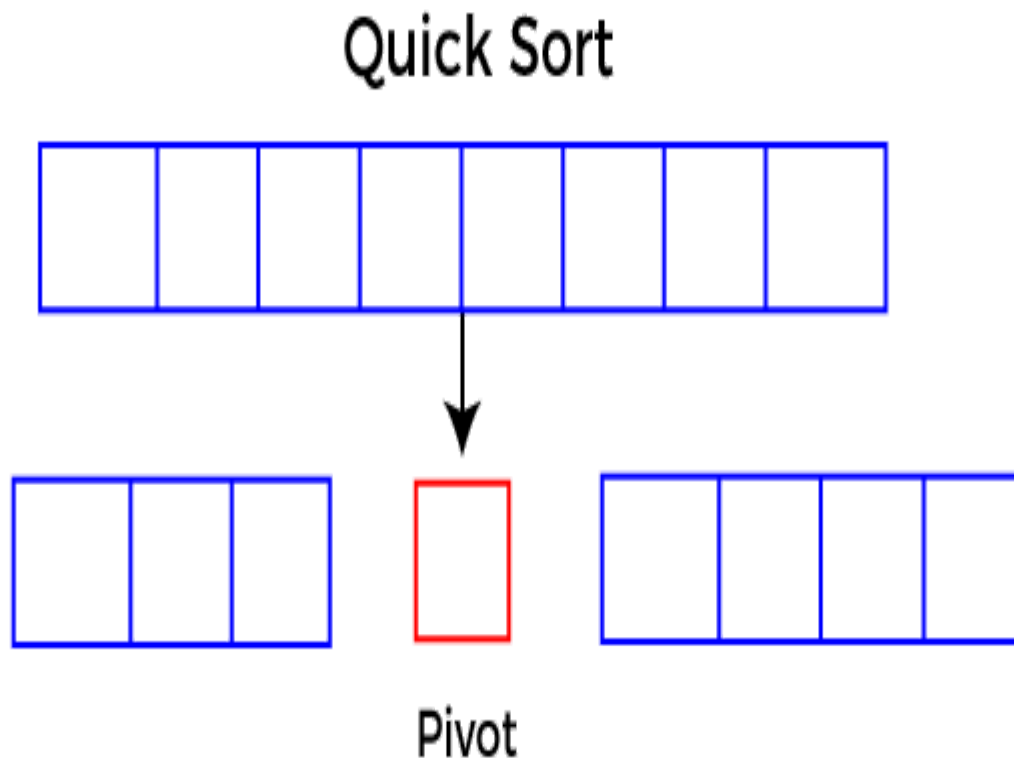
After that, left and right sub-arrays are also partitioned using the same approach. It will continue until the single element remains in the sub-array.

Choosing the pivot

Picking a good pivot is necessary for the fast implementation of quicksort. However, it is typical to determine a good pivot. Some of the ways of choosing a pivot are as follows –

- Pivot can be random, i.e. select the random pivot from the given array.
- Pivot can either be the rightmost element or leftmost element of the given array.

Select median as the pivot element.



Quick Sort Function Algorithm

1. QUICKSORT (array A, start, end)

```

2.  {
3.  if (start < end)
4.  {
5.  p = partition(A, start, end)
6.  QUICKSORT (A, start, p - 1)
7.  QUICKSORT (A, p + 1, end)
8.  }
9.  }

```

PARTITION (array A, start, end)

```

10. {
11. 1 pivot = A[end]
12. 2 i = start-1
13. 3 for j = start to end -1 {
14. 4 do if (A[j] < pivot) {
15. 5 then i = i + 1
16. 6 swap A[i] with A[j]
17. 7 }}
18. 8 swap A[i+1] with A[end]
19. 9 return i+1
20. }

```

Time Complexity:

Best Case: $\Omega(N \log(N))$

Average Case: $\theta(N \log(N))$

Worst Case: $O(N^2)$

Auxiliary Space: $O(1)$, if we don't consider the recursive stack space. If we consider the recursive stack space then, in the worst case quicksort could make $O(N)$.

Quick sort is stable sort and inplace sort.

Merge Sort

Merge sort is similar to the quick sort algorithm as it uses the divide and conquer approach to sort the elements. It is one of the most popular and efficient sorting algorithm. It divides the given list into two equal halves, calls itself for the two halves and then merges the two sorted halves.

The sub-lists are divided again and again into halves until the list cannot be divided further.

I Merge-Sort (A, p, r)
if $p < r$
then $q \leftarrow (p + r) / 2$
Merge-Sort (A, p, q)
Merge-Sort ($A, q+1, r$)
Merge (A, p, q, r)

II Merge (A, p, q, r)
 $n_1 \leftarrow q - p + 1$
 $n_2 \leftarrow r - q$
create array $L[1 \dots n_1+1]$ and $R[1 \dots n_2+1]$
for $i \leftarrow 1$ to n_1
do $L[i] \leftarrow A[p+i-1]$
for $j \leftarrow 1$ to n_2
do $R[j] \leftarrow A[q+j]$
 $L[n_1+1] \leftarrow \infty$
 $R[n_2+1] \leftarrow \infty$
 $i \leftarrow 1$
 $j \leftarrow 1$
for $k \leftarrow p$ to r
do if $L[i] \leq R[j]$
then $A[k] \leftarrow L[i]$
 $i \leftarrow i+1$
else $A[k] \leftarrow R[j]$
 $j \leftarrow j+1$

Time Complexity:
- (Worst, Best, Average):
- Not In place Sorting algorithm
- Stable Sort

1	2	3	4	5	6	7	8
5	10	5	20	25	30	40	35
p			q				u

$$p=1; u=8$$

$$1 < 8 \quad \text{then } q = \lfloor \frac{1+8}{2} \rfloor = \lfloor \frac{9}{2} \rfloor = 4$$

$$q=4$$

Merge Sort(A, 1, 4)

Merge Sort(A, 5, 8)

Merge(A, 1, 4, 8)

Merge Sort(A, 1, 4)

$$p=1, u=4$$

$$1 < 4 \quad \text{so } q = \lfloor \frac{1+4}{2} \rfloor = \lfloor \frac{5}{2} \rfloor = 2$$

$$q=2$$

Merge Sort(A, 1, 2)

Merge Sort(A, 3, 4)

Merge(A, 1, 2, 4)

Merge Sort(A, 1, 2)

$$p=1, u=2$$

$$1 < 2 \quad \text{then } q = \lfloor \frac{1+2}{2} \rfloor = 1$$

Merge Sort(A, 1, 1)

Merge Sort(A, 2, 2)

Merge Sort ($A, 1, 1$)
1 1 No change

then Call Merge Sort ($A, 2, 2$)
Here 2 2 No change

call Merge ($A, 1, 1, 2$)

$p=1, q=1, w=2$

$n_1 = q - p + 1 = 1 - 1 + 1 = 1$

$n_2 = 2 - 1 = 1$

Create array $L[1..2]$ and $R[1..2]$

for $i=1$ to 1

$L[i] = A[i+1-1]$

$L[1] = A[1]$

$A[]$

15	10
----	----

$L[1] = 15$

for $j=1$ to 1

$R[j] = A[i+j]$

$R[1] = 10$

$R[1] = A[2]$

$L[2] = \infty$ and $i=1$

$R[2] = \infty$ $j=1$

For $k=1$ to 2

$k=1$

if $L[i] \leq R[j]$

$15 \leq 10$ (false)

$A[i] = R[j]$ i.e. 10

$j = 1 + 1 = 2$

$k=2$ and $j=2$

$L[2] \leq R[2]$

1	2	3	4	5	6	7	8
15	10	5	20	25	30	40	35
p			q				u

$$p=1; u=8$$

$$1 < 8 \text{ then } q = \lfloor \frac{1+8}{2} \rfloor = \lfloor \frac{9}{2} \rfloor = 4$$

$$q=4$$

Merge Sort(A, 1, 4)

Merge Sort(A, 5, 8)

Merge(A, 1, 4, 8)

Merge Sort(A, 1, 4)

$$p=1, u=4$$

$$1 < 4 \text{ so } q = \lfloor \frac{1+4}{2} \rfloor = \lfloor \frac{5}{2} \rfloor = 2$$

$$q=2$$

Merge Sort(A, 1, 2)

Merge Sort(A, 3, 4)

Merge(A, 1, 2, 4)

Merge Sort(A, 1, 2)

$$p=1, u=2$$

$$1 < 2 \text{ then } q = \lfloor \frac{1+2}{2} \rfloor = 1$$

Merge Sort(A, 1, 1)

Merge Sort(A, 2, 2)

Merge Sort (A, 1, 1)
1 < 1 No change

then Call Merge Sort (A, 2, 2)
Here 2 < 2 No change

call Merge (A, 1, 1, 2)

$p=1, q=1, r=2$

$m_1 = q - p + 1 = 1 - 1 + 1 = 1$

$m_2 = 2 - 1 = 1$

Create array L[1..2] and R[1..2]

for $i=1$ to 1

$L[i] = A[i+1-1]$

$L[1] = A[1]$

A[]

15	10
----	----

$L[1] = 15$

for $j=1$ to 1

$R[j] = A[i+1]$

$R[1] = 10$

$R[1] = A[2]$

$L[2] = \infty$ and $i=1$

$R[2] = \infty$ $j=1$

For $k=1$ to 2

$k=1$

if $L[i] \leq R[j]$

$15 \leq 10$ (false)

$A[i] = R[j]$ i.e. 10

$j = 1 + 1 = 2$

$k=2$ and $j=2$

$L[2] \leq R[2]$

$$15 \leq \infty \text{ (true)}$$

$$i = \text{start} \quad H1 = 2 \quad A[2] = 15$$

$$\boxed{10 \mid 15}$$

Merge-Sort(A, 1, 2) is completed.

then call Merge-Sort(A, 3, 4)

$$p = 3 \quad q = 4$$

$$3 < 4 \text{ then } r = \left\lfloor \frac{p+q}{2} \right\rfloor = \left\lfloor \frac{3+4}{2} \right\rfloor = 3$$

call Merge-Sort(A, 3, 3)

No change

Merge-Sort(A, 4, 4)

No change

Merge-Sort(A, 3, 3, 4)

call Merge(A, 3, 3, 4)

$$p = 3, \quad q = 3, \quad r = 4$$

$$n_1 = 1, \quad m_2 = 1$$

Create array L and R

for $i = 1$ to 1

$$L[1] = A[3+1-1]$$

$$L[1] = A[3] = 5$$

for $j = 1$ to 1

$$R[1] = A[3+1]$$

$$R[1] = 20$$

$$\text{i.e. } L[2] = \infty \text{ and } R[2] = \infty$$

$$i = 1 \text{ and } j = 1$$

Now for $k=3$ to 4

$k=3, i=1, j=1$

$L[1] = 5$

~~$R[1]$~~ $R[1] = 20$

$5 \leq 20$ (True)

$A[3] = 5$ and $i = 1+1 = 2$

$k=4, i=2, j=1$

$L[2] = \infty$ $R[1] = 20$

$L[2] \not\leq R[1]$

$A[4] = 20$

1	2	3	4
0	15	5	20

Merge $(A, 1, 2, 4)$

$p=1, q=2, n=4$

$n_1 = 2-1+1 = 2$

$n_2 = 4-2 = 2$

Create array $L[1..3]$ and $R[1..3]$

for $i=1$ to 2

$i=1$ $L[1] = A[1+1-1]$

$L[1] = A[1] = 10$

$i=2$ $L[2] = A[1+2-1]$

$= A[2]$

$L[2] = 15$

$L[]$

1	2	3
10	15	∞

for $j=1$ to 2

$R[1] = A[2+1] = 5$

$R[2] = A[2+2] = 20$

1	2	3
5	20	∞

Now $i=1; j=1$

For $k=1$ to 4

$k=1$ $L[1] \leq R[1]$ i.e. $10 \leq 5$ (false)
So $A[1] = R[1]$ i.e. $A[1] = 5$ and $j = 1+1 = 2$

$k=2$ $L[1] \leq R[2]$ i.e. $10 \leq 20$ (True)
So $A[2] = L[1]$ i.e. $A[2] = 10$ and $i = 1+1 = 2$

$k=3$ $L[2] \leq R[2]$ i.e. $15 \leq 20$ (True)
So $A[3] = L[2]$ i.e. $A[3] = 15$ and $i = 2+1 = 3$

$k=4$ $L[3] \leq R[2]$ i.e. $0 \leq 20$ (True)
So $A[4] = R[2]$ i.e. $A[4] = 20$ and $j = 2+1 = 3$
now array is

5	10	15	20
---	----	----	----

Now call Merge-Sort ($A, 5, 8$)

$p=5, r=8$
 $s < r$ $q = \left\lfloor \frac{5+8}{2} \right\rfloor = 6$

then call Merge-Sort ($A, 5, 6$)
Merge-Sort ($A, 7, 8$)
Merge ($A, 5, 6, 8$)

Merge-Sort ($A, 5, 6$)

$p=5, r=6$
if $s < r$ then $q = \left\lfloor \frac{5+6}{2} \right\rfloor = 5$

then call Merge-Sort(A, 5, 5)
Merge-Sort(A, 6, 6)
Merge(A, 5, 5, 6)

call Merge-Sort(A, 5, 5)
 $5 \leq 5$ false No change

Merge-Sort(A, 6, 6)
 $6 \leq 6$ false No change

call Merge(A, 5, 5, 6)
 $p=5, q=5, r=6$
 $m_1=1$
 $m_2=1$

Create array L[1...2] and R[1...2]

$L[1] = A[5]$ i.e. $L[1] = 25$

$R[1] = A[6]$ i.e. $R[1] = 30$

$L[2] = \infty$

$R[2] = \infty$

$L[] = \begin{array}{c|c} 1 & 2 \\ \hline 25 & \infty \end{array}$

$R[] = \begin{array}{c|c} 1 & 2 \\ \hline 30 & \infty \end{array}$

$i=1$ and $j=1$

For $k=5$ to 6

$k=5$ $L[1] \leq R[1]$ i.e. $25 \leq 30$ (True)
so $A[5] = 25$ and $i = i+1 = 2$

$k=6$ $L[2] \leq R[2]$ i.e. $\infty \leq 30$ (False)
so $A[6] = 30$ and $j = j+1 = 2$

How array is

	1	2	3	4	5	6
A[]	5	10	15	20	25	30

Merge-Sort (A, 7, 8)

A[]	5	10	15	20	25	30	35	40
------	---	----	----	----	----	----	----	----

Merge
~~Merge-Sort~~ (A, 5, 6, 8)

A[]	5	10	15	20	25	30	35	40
------	---	----	----	----	----	----	----	----

Merge A(A, 1, 4, 8)

	1	2	3	4	5	6	7	8
	5	10	15	20	25	30	35	40

Case	Time Complexity
Best Case	$O(n \cdot \log n)$
Average Case	$O(n \cdot \log n)$
Worst Case	$O(n \cdot \log n)$

Space Complexity	$O(n)$
Stable	YES

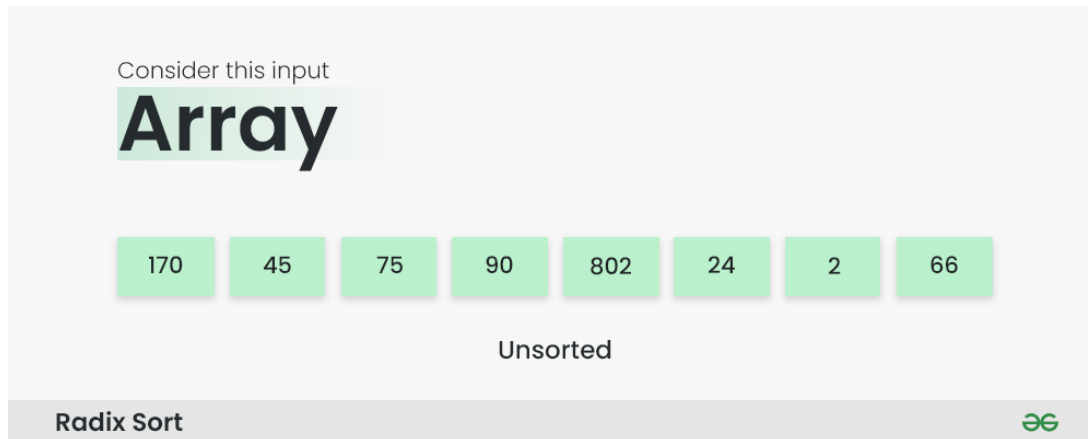
Merge sort is a stable sort and not in place sort

Radix Sort Algorithm

The key idea behind Radix Sort is to exploit the concept of place value. It assumes that sorting numbers digit by digit will eventually result in a fully sorted list. Radix Sort can be performed using different variations, such as Least Significant Digit (LSD) Radix Sort or Most Significant Digit (MSD) Radix Sort.

How does Radix Sort Algorithm work?

To perform radix sort on the array [170, 45, 75, 90, 802, 24, 2, 66], we follow these steps:



Consider this input

Array

170	45	75	90	802	24	2	66
-----	----	----	----	-----	----	---	----

Unsorted

Radix Sort 

The diagram illustrates the initial state of an array for Radix Sort. It features a light gray background with the text 'Consider this input' above a large, bold, black 'Array' label. Below the label, eight green rectangular boxes are arranged horizontally, each containing a number: 170, 45, 75, 90, 802, 24, 2, and 66. These boxes are labeled 'Unsorted' below them. At the bottom of the diagram, there is a dark gray bar with the text 'Radix Sort' on the left and a small green logo on the right.

How does Radix Sort Algorithm work | Step 1

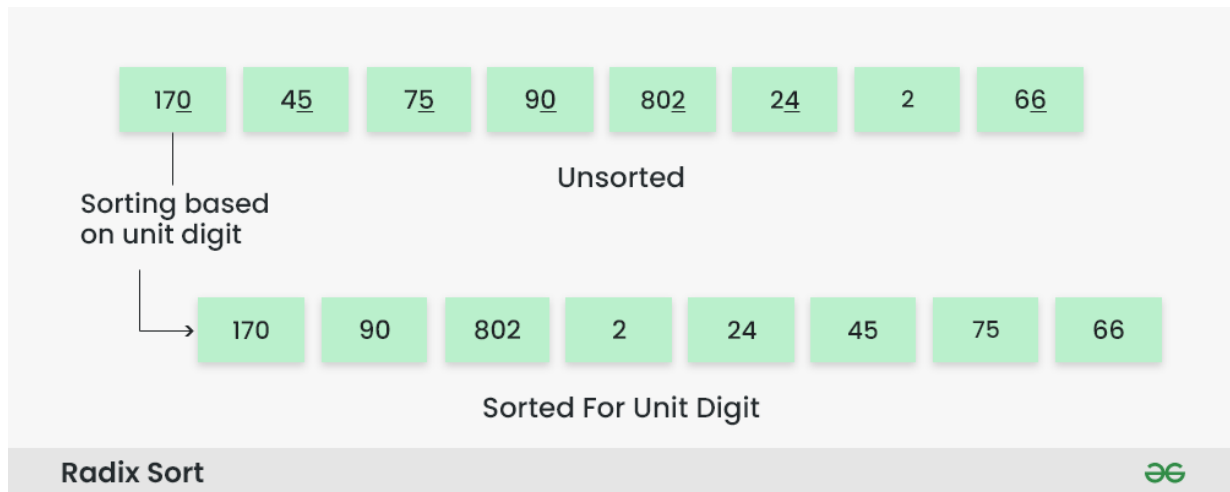
Step 1: Find the largest element in the array, which is 802. It has three digits, so we will iterate three times, once for each significant place.

Step 2: Sort the elements based on the unit place digits ($X=0$). We use a stable sorting technique, such as counting sort, to sort the digits at each significant place. It's important to understand that the default implementation of counting sort is unstable i.e. same keys can be in a different order than the input array. To solve this problem, We can iterate the input array in reverse order to build the output array. This strategy helps us to keep the same keys in the same order as they appear in the input array.

Sorting based on the unit place:

Perform counting sort on the array based on the unit place digits.

The sorted array based on the unit place is [170, 90, 802, 2, 24, 45, 75, 66].



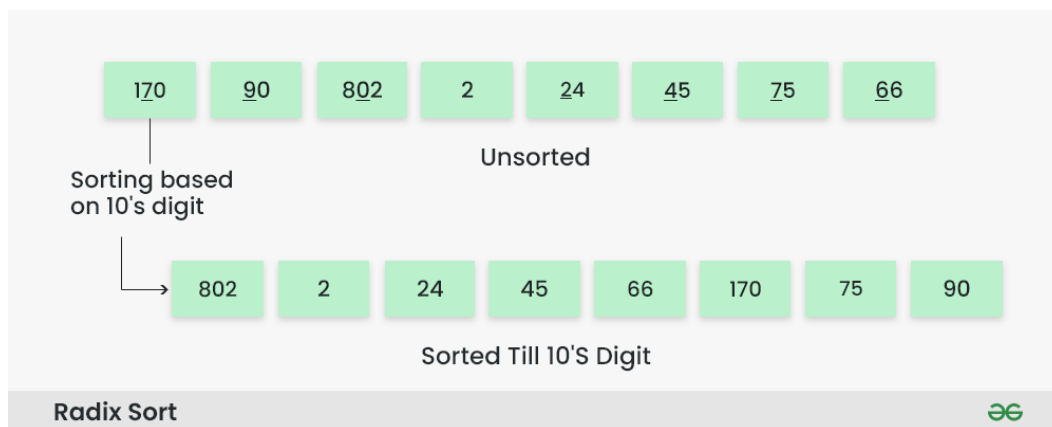
How does Radix Sort Algorithm work | Step 2

Step 3: Sort the elements based on the tens place digits.

Sorting based on the tens place:

Perform counting sort on the array based on the tens place digits.

The sorted array based on the tens place is [802, 2, 24, 45, 66, 170, 75, 90].



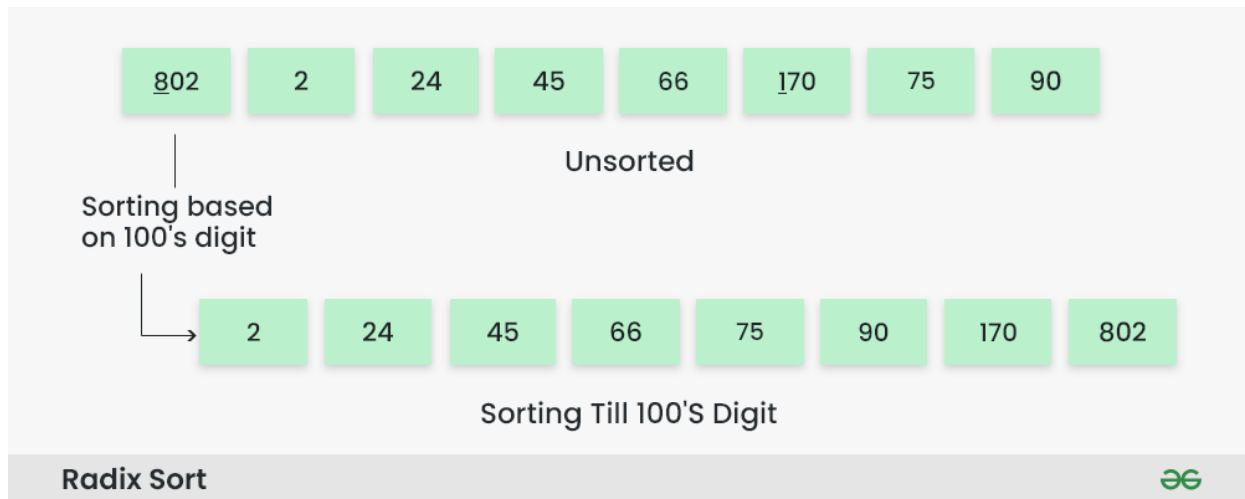
How does Radix Sort Algorithm work | Step 3

Step 4: Sort the elements based on the hundreds place digits.

Sorting based on the hundreds place:

Perform counting sort on the array based on the hundreds place digits.

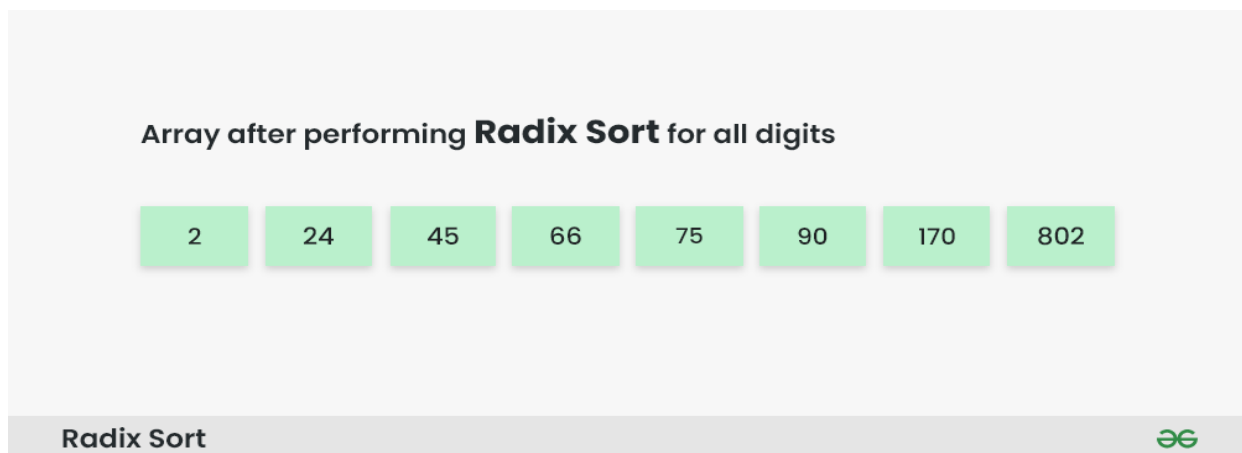
The sorted array based on the hundreds place is [2, 24, 45, 66, 75, 90, 170, 802].



How does Radix Sort Algorithm work | Step 4

Step 5: The array is now sorted in ascending order.

The final sorted array using radix sort is [2, 24, 45, 66, 75, 90, 170, 802].



Complexity Analysis of Radix Sort:

Time Complexity:

Radix sort is a non-comparative integer sorting algorithm that sorts data with integer keys by grouping the keys by the individual digits which share the same significant position and value. It has a time complexity of $O(d * (n + b))$, where d is the number of digits, n is the number of elements, and b is the base of the number system being used.

In practical implementations, radix sort is often faster than other comparison-based sorting algorithms, such as quicksort or merge sort, for large datasets, especially when the keys have many digits. However, its time complexity grows linearly with the number of digits, and so it is not as efficient for small datasets.

Auxiliary Space:

Radix sort also has a space complexity of $O(n + b)$, where n is the number of elements and b is the base of the number system. This space complexity comes from the need to create buckets for each digit value and to copy the elements back to the original array after each digit has been sorted.

Radix sort is a stable sorting algorithm but not an in-place sorting algorithm:

Heap Sort

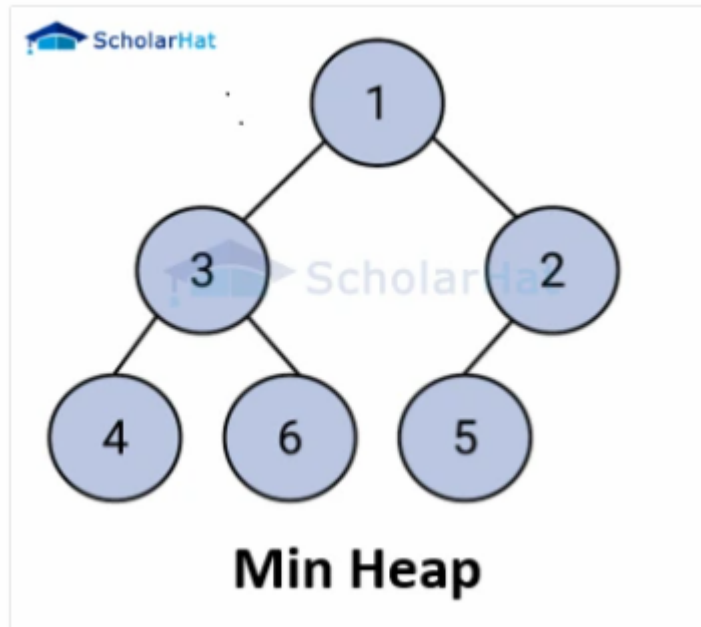
What is a Heap in Data Structures?

A heap is a tree-like data structure in which the tree is a complete binary tree that satisfies the heap property. According to the heap property, all the children of a given node must be greater than the parent node, or all the children must be smaller than the parent node. This type of data structure is also called a binary heap.

Types of Heap Data Structure

There are two main types of heap data structures:

1. **Max Heap:** In this, all the nodes (including the root) are greater than their respective child nodes. The key of the root node is always the largest among all other nodes.
- 2.
3. **Min Heap:** In this, all the nodes (including the root) are smaller than their respective child nodes. The key of the root node is always the smallest among all other nodes.



What is Heap Sort in Data Structures?

Heap sort is a comparison-based, in-place sorting algorithm that visualizes the elements of the array in the form of a heap data structure to sort it. It divides the input array into two parts; sorted region, and unsorted region. The sorted region is initially empty, while the unsorted region contains all the elements. The largest element from the unsorted region is picked iteratively and added to the sorted region. The algorithm arranges the unsorted region in a heap.

Working of Heap Sort Algorithm

1. Build Max Heap: Create a max heap by visualizing all the elements of the array in a binary tree. This process ensures that the largest element is at the root of the heap.
- 2.Repeat the following steps until the heap contains only one element:
 - 1.Swap: Remove the root element and put it at the end of the array (nth position). Put the last item of the tree (heap) in the vacant place.
 - 2.Remove: Reduce the size of the heap by 1.
 - 3.Heapify: Heapify the root element again so that we have the highest element at the root.
- 3.Obtain Sorted Array: The sorted array is obtained by reversing the order of the elements in the input array.

Build Max-Heap

Heapify

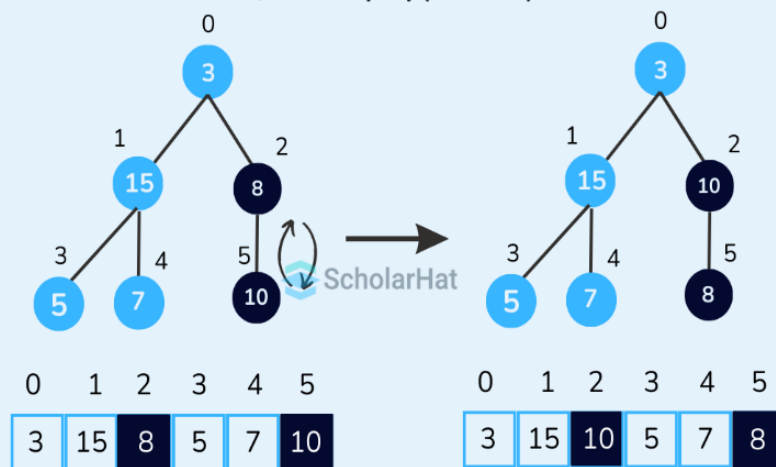
arr 0 1 2 3 4 5
 3 15 8 5 7 10

n = 6

i = 6/2-1=2 # loop runs from 2 to 0

Create array and calculate i

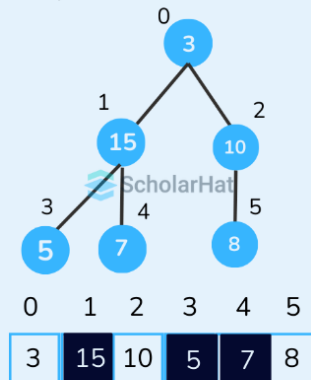
i = 2 → heapify(arr,6,2)



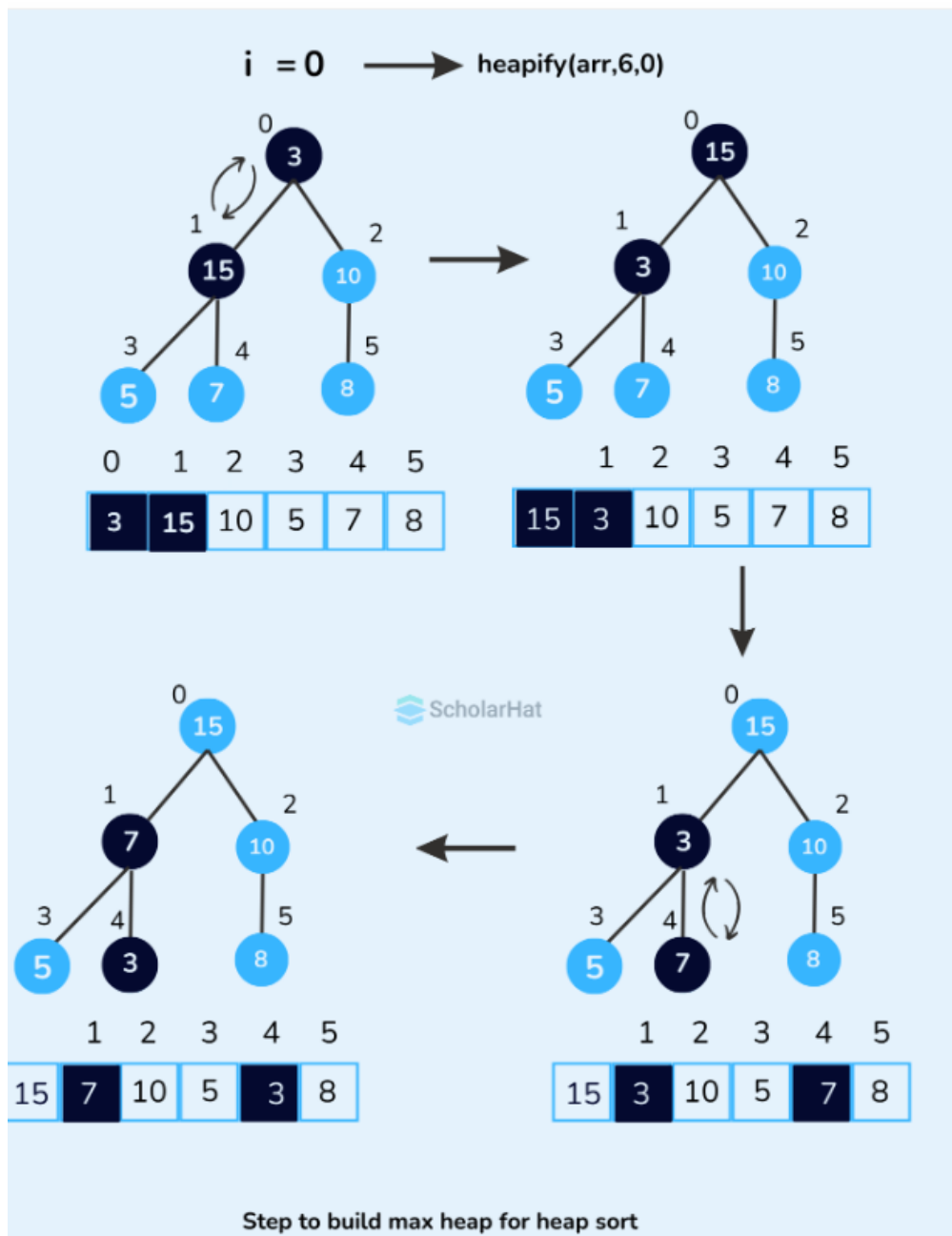
Steps to build max heap for heap sort

i = 1 → heapify(arr,6,1)

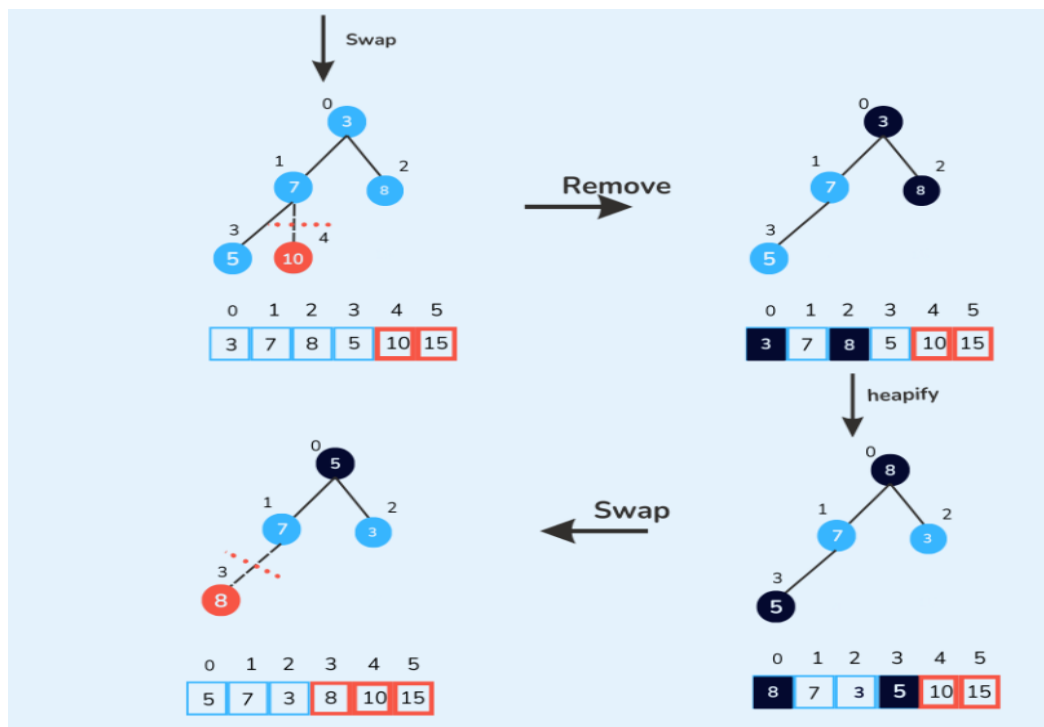
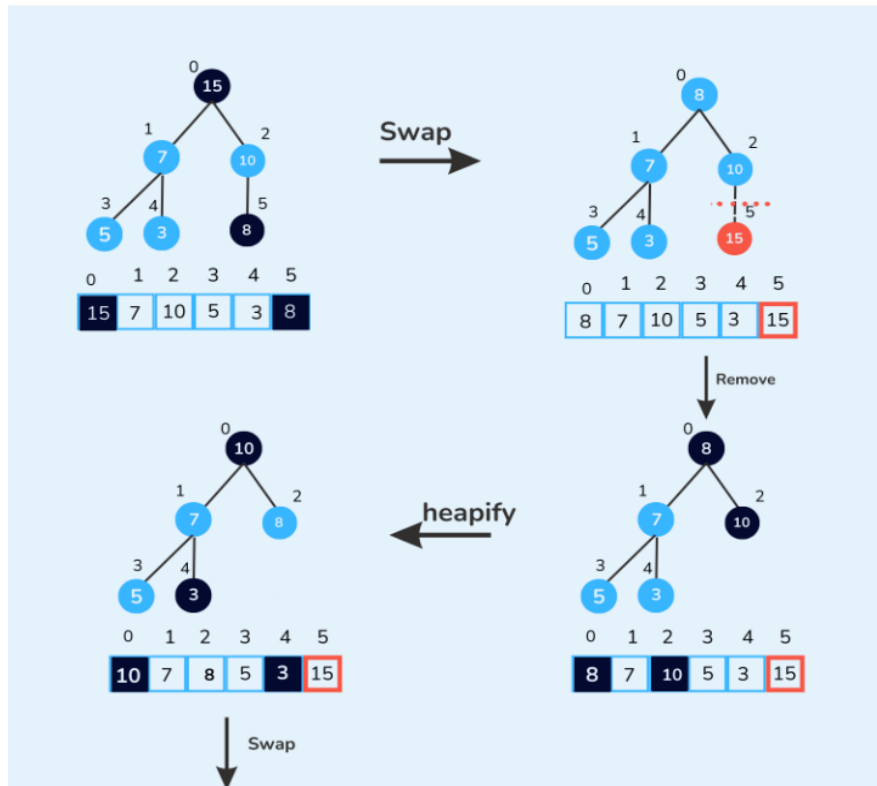
Already heapified

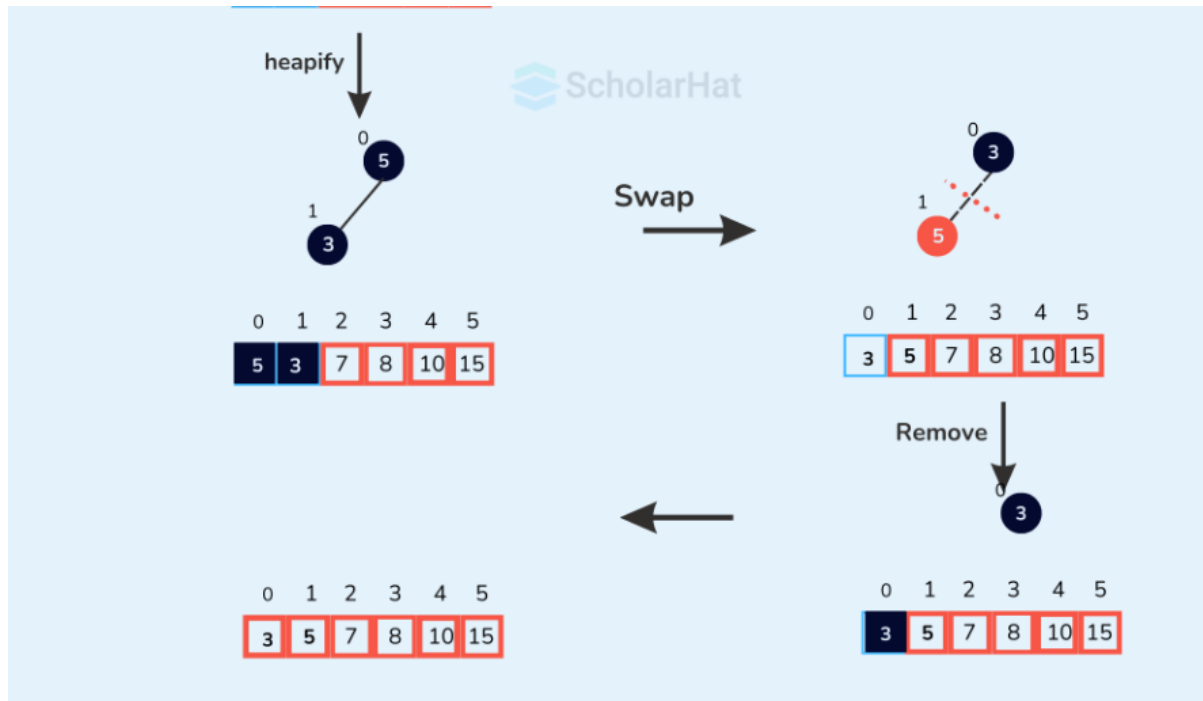
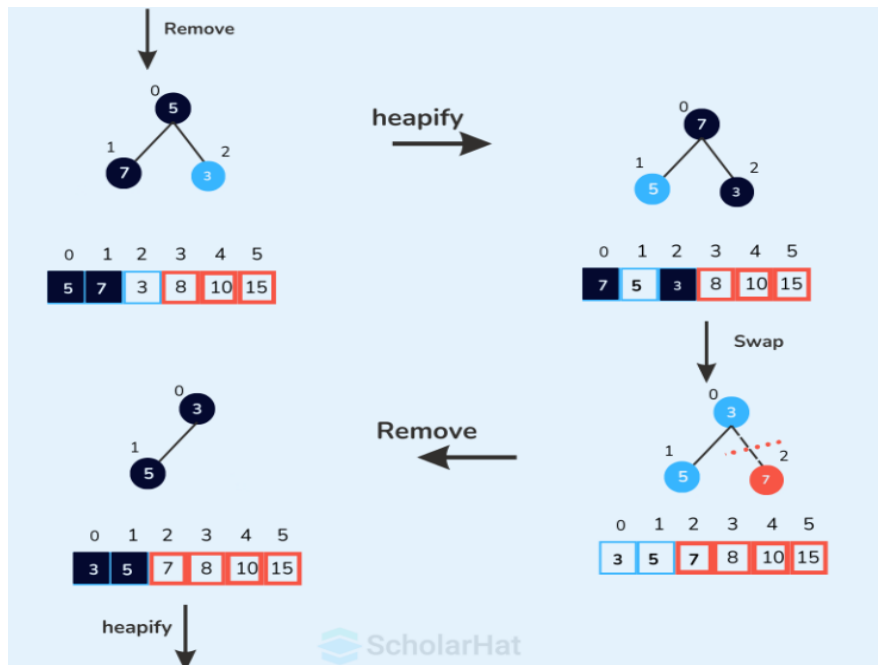


Steps to build max heap for heap sort



Swap, Remove, Heapify





Complexity Analysis of Heap Sort Algorithm

1. Time Complexity

Case	Time Complexity
Best	$O(n \log n)$
Worst	$O(n \log n)$
Average	$O(n \log n)$

2. **Space Complexity:** The space complexity is $O(1)$ because we have a fixed number of variables, and we do not need any extra memory space apart from the loop variables and auxiliary variables that include temp, n, index, and largest.

Heap sort is an in-place sorting algorithm but it is not stable